

Traffic Sign Condition Monitor

This repository contains the training pipeline and deployment strategy for a real-time computer vision model designed to monitor traffic sign infrastructure. Built using **YOLOv8**, the model processes live dashcam feeds from government buses to automatically detect and classify traffic signs as either Good or Damaged, streamlining maintenance alerts.

Project Overview

Maintaining road infrastructure relies heavily on manual reporting. This project automates the detection of defective signage (graffiti, faded, bent, or occluded signs) using edge AI. By deploying this model on moving buses, transit authorities can continuously audit city-wide infrastructure without dedicated surveying vehicles.

Dataset Strategy & Preprocessing

The model was trained on a custom unified dataset, merging pristine signs with various defect states.

- **Source 1:** IRTSD Dataset (Manually annotated pristine signs).
- **Source 2:** Roboflow Damaged Traffic Sign Dataset.
- **Class Mapping (Ontology):** Disparate raw classes were consolidated into a strict binary format using Roboflow's Preprocessing Pipeline to optimize decision-making on the edge.
 - **Mapped to Good:** healthy, ok
 - **Mapped to Damaged:** deformation, dirty, graffiti, knocked, occluded, perforation, stickers, worn
- **Processing:** All images resized to 640x640 (Fit with black edges) with auto-orientation enabled.

Model Training (Google Colab)

The model was trained using the Ultralytics YOLOv8 architecture on a Google Colab T4 GPU instance, utilizing Google Drive for persistent checkpoint storage to prevent data loss.

Training Configurations:

- **Architecture:** YOLOv8 Small (yolov8s.pt)
- **Epochs:** 50
- **Batch Size:** 16
- **Image Size:** 640

Python Training Script:

```
from ultralytics import YOLO
```

```
# Load the YOLOv8 small model
```

```
model = YOLO('yolov8s.pt')

# Execute training with Drive backup
model.train(
    data='/content/dataset/data.yaml',
    epochs=50,
    imgsz=640,
    batch=16,
    project='/content/drive/MyDrive/traffic_model'
)
```

(Note: To prevent GitHub .ipynb rendering bugs, cell outputs were cleared prior to committing the notebook).

Edge Deployment Pipeline (Live Dashcam Feed)

Moving from static images to a vibrating, moving bus requires specific edge optimizations:

1. **Hardware:** Target deployment on fanless edge hardware (e.g., NVIDIA Jetson Orin Nano or Intel Industrial PCs with AI accelerators).
2. **Model Optimization:** The resulting best.pt weights are exported to **TensorRT** (for NVIDIA) or **ONNX** to maximize FPS and reduce thermal load.
3. **Object Tracking:** Integration with **ByteTrack** to assign unique IDs to detected signs, preventing the database from logging the same damaged sign 30 times a second.
4. **Telemetry:** Asynchronous API calls sync bounding box detections with the bus's onboard GPS module, batch-uploading location data over cellular networks.

Future Roadmap: MTSVD Integration

To further stabilize the model against the realities of 25-30fps bus dashcams, future iterations will integrate the **MTSVD (Missing Traffic Signs Video Dataset)**.

- Injecting motion-blurred sequential frames to fine-tune the tracker.
- Training a secondary contextual network to identify where signs *should* exist (based on road layout cues) to flag completely missing/stolen infrastructure.

Usage

To test the weights on a local image:

```
pip install ultralytics
yolo task=detect mode=predict model=weights/best.pt source=test_image.jpg show=True
```